

AGRO-GPT: A MULTIMODAL AGRICULTURAL DISEASE UNDERSTANDING SYSTEM

A Project Report

Submitted by

Ansuman Mantri (Reg. No: 2241013098)

Aniket Nayak (Reg. No: 2241014056)

Dipankar Mohapatra (Reg. No: 2241013403)

Mrutyunjaya Tripathy (Reg. No: 2241020033)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

(Specialization in Artificial Intelligence and Machine Learning)



**Centre for Artificial Intelligence and Machine Learning
Department of Computer Science and Engineering**

**Faculty of Engineering and Technology
Institute of Technical Education and Research**

**Siksha 'O' Anusandhan
(Deemed to be University)**

Bhubaneswar, Odisha, India

June, 2026

CERTIFICATE



This is to certify that the project report titled “**AGROGPT: A MULTIMODAL AGRICULTURAL DISEASE UNDERSTANDING SYSTEM**” being submitted by **Mr. Ansuman Mantri, Mr. Aniket Nayak, Mr. Dipankar Mohapatra, and Mr. Mrutyunjaya Tripathy** of **Section - 2241012** to the **Institute of Technical Education and Research, Siksha ‘O’ Anusandhan (Deemed to be University)**, Bhubaneswar for the partial fulfillment for the degree of **Bachelor of Technology in Computer Science and Engineering (Specialization in Artificial Intelligence and Machine Learning)** is a record of original confide work carried out by them under our supervision and guidance. The project work, in my opinion, has reached the requisite standard fulfilling the requirements for the Degree of **Bachelor of Technology**.

The results contained in this project work have not been submitted in part or full to any other University or Institute for the award of any degree or diploma.

Ms. Chinmayee Nayak

Associate Professor

Centre for Artificial Intelligence and Machine Learning

Department of Computer Science & Engineering

Faculty of Engineering & Technology, Institute of Technical Education and Research

Siksha ‘O’ Anusandhan (Deemed to be University), Bhubaneswar

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported and guided us throughout the successful completion of our Senior Year Project titled “**AGROGPT: A MULTI-MODAL AGRICULTURAL DISEASE UNDERSTANDING SYSTEM**”.

First and most importantly, we would like to extend our thanks and sincerest gratitude to our well-regarded project supervisor, **Ms. Chinmayee Nayak** for her continuous guidance, other assistance, reliable advice, and helpful suggestions throughout this project and for motivating us to improve our work in the technical and documentation parts.

And we would like to thank our university for giving us the opportunity to work on such a creative and practical agricultural machinery project with AI. The knowledge and skills that we have gained during this project will make us much more proficient in the areas of deep learning, explainable AI, backend development, cloud deployment, and intelligent agricultural systems.

Name of Student	Roll No.	Signature
Student 1	Reg. No.	_____
Student 2	Reg. No.	_____
Student 3	Reg. No.	_____
Student 4	Reg. No.	_____

Date: _____

Place: _____

DECLARATION

We declare that this written submission represents our ideas in our own words and where other's ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/fact/source in our submission. We understand that any violation of the above will cause for disciplinary action by the University and can also evoke penal action from the sources which have not been properly cited or from whom proper permission has not been taken when needed.

Name of Student

Roll No.

Signature

Student 1

Reg. No.

Student 2

Reg. No.

Student 3

Reg. No.

Student 4

Reg. No.

Date: _____

Place: _____

REPORT APPROVAL

This project report titled “**AGROGPT: A MULTIMODAL AGRICULTURAL DISEASE UNDERSTANDING SYSTEM**” submitted by **Mr. Ansuman Mantri, Mr. Aniket Nayak, Mr. Dipankar Mohapatra, and Mr. Mrutyunjaya Tripathy** is approved for the degree of **Bachelor of Technology in Computer Science and Engineering (Specialization in Artificial Intelligence and Machine Learning)**.

Examiner(s)

Supervisor

Project Coordinator(s)

Chief Coordinator, CAIML

PREFACE

Agriculture is one of the most important industries affecting food production and economic growth. But plant diseases continue to affect crop quality and agricultural production, causing huge monetary losses to the farmers. Accurate and early disease identification is a prerequisite for improving crop management and reducing agricultural losses.

“AGROGPT: A MULTIMODAL AGRICULTURAL DISEASE UNDERSTANDING SYSTEM” aims at developing a smart agricultural service to diagnose plant disease through explainable AI and deep learning. The system uses Grad-CAM explainability as a way to visually interpret model predictions and transfer learning based on EfficientNet-B0 for illness categorization. Uncertainty-aware prediction logic is also used in research for increased confidence estimates and prediction accuracy.

The system has an AI-based agriculture knowledge engine that gives the user suggestions on diseases, doctor diagnoses, treatments, and avoidance recommendations as well as disease predictions. The frontend interface is designed to work with contemporary web technology and cloud deployment services like Hugging Face Spaces and Vercel to make it convenient. The backend service is built upon FastAPI.

The overall workflow of the project is Preparing the data, conducting exploratory data analysis, applying the appropriate data preprocessing, training the model, designing the architecture, factorizing the explainability, how to deploy, and performance assessment of this project are presented in this paper.

Contents

LIST OF FIGURES	iii
LIST OF TABLES	iv
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Project Overview / Problem Statement	2
1.3 Motivations and Objectives	3
1.4 Scope and Significance of the Work	5
1.5 Organisation of the Report	6
2 REVIEW OF LITERATURE	8
2.1 Review of Existing Systems / Related Work	8
2.2 Research Gaps and Problem Identification	9
3 MATERIALS AND METHODS	11
3.1 Dataset Description / / Input Specification	11
3.2 System Architecture / Block Diagram / UML Diagram / DFD / Use Case Diagram	13
3.3 Methodology / Algorithm / Model Description	15
3.3.1 Algorithm Used	16
3.3.2 Model Description	16
3.4 Tools, Technologies and Frameworks Used	18
3.5 Performance Metrics / Evaluation Criteria	18
3.5.1 Evaluation Criteria	19
4 EXPERIMENTATION AND RESULTS	20
4.1 System Specifications	20
4.2 Parameter Settings / Configuration Details	21
4.3 Implementation Details / Module Description	22
4.3.1 Dataset and Preprocessing Module	22
4.3.2 Disease Classification Module	23
4.3.3 Uncertainty-Aware Prediction Module	23

4.3.4	Grad-CAM Explainability Module	24
4.3.5	Knowledge Engine Module	24
4.3.6	Backend API Module	24
4.3.7	Frontend Interface Module	25
4.3.8	Deployment Module	26
4.4	Results and Outputs	27
4.5	Result Analysis and Validation	28
5	ETHICAL, SOCIAL & SUSTAINABILITY CONSIDERATIONS	31
5.1	Ethical Consideration	31
5.2	Social Impact and Societal Relevance	32
5.3	Sustainability Consideration	32
6	CONCLUSION AND FUTURE SCOPES	34
6.1	Summary of Findings	34
6.2	Limitations of the Work	35
6.3	Scope for Future Enhancements	36
	REFERENCES	38
	APPENDIX	40
	Project Proposal	40
	Similarity Check Report	41
	Team Contribution	42
	Course Outcomes (COs), Program Outcomes (POs) and Program Specific Outcomes (PSOs)	43
	Evidences Linked to CO–PO & PSO Mapping	45
	Code Structure	48
	Code Listings	50
6.3.1	Data Preprocessing and Augmentation	50
6.3.2	EfficientNet-B0 Model Training	51
6.3.3	Prediction and Uncertainty-Aware Logic	52
6.3.4	Grad-CAM Explainability Module	53
6.3.5	FastAPI Backend Integration	54

List of Figures

3.1	Dataset Composition Visualization	12
3.2	t-SNE Visualization of Dataset Feature Distribution	13
3.3	Overall System Architecture of AGRO-GPT	14
3.4	Overall Architecture of the AGRO-GPT System	15
4.1	FastAPI Backend Execution and Hugging Face Deployment Logs	25
4.2	Frontend Interface of the AGRO-GPT System	26
4.3	Cloud Deployment of AGRO-GPT using Vercel	26
4.4	Prediction Output and Grad-CAM Explainability Visualization	28
4.5	Validation Accuracy Trend During Training	29
4.6	Loss vs Accuracy Relationship	29
4.7	Grad-CAM Heatmap Visualization of Diseased Leaf	30
4.8	Confusion Matrix of Plant Disease Classification	30
6.1	Code Structure Overview of AGRO-GPT System	48
6.2	Data Preprocessing and Augmentation Pipeline	50
6.3	EfficientNet-B0 Training Implementation	51
6.4	Prediction and Confidence Evaluation Logic	52
6.5	Grad-CAM Heatmap Generation Function	53
6.6	FastAPI Backend Prediction API	54

List of Tables

1.1	Challenges Faced in Traditional Disease Detection	4
2.1	Literature Review Summary	9
2.2	Comparative Analysis of Existing Deep Learning Models	10
3.1	Dataset Description and Statistics	11
3.2	Sample Dataset Distribution	12
3.3	System Architecture Modules	14
3.4	Image Augmentation Techniques Used	16
3.5	Methodology Workflow	17
3.6	Tools and Technologies Used	18
3.7	Performance Evaluation Results	19
4.1	System Specifications	20
4.2	Hyperparameter Tuning Experiments	21
4.3	Parameter Settings and Configuration Details	22
4.4	Performance Metrics of AGRO-GPT	27
4.5	Comparison with Existing Approaches	29

Chapter 1

INTRODUCTION

1.1 Introduction

Agriculture is For sure fundamental to the sustenance of human life. It brings food, drives the economy, and creates jobs. For instance, in India, a considerable percentage of the population is engaged in farming either directly or indirectly. A lot, crop production determines the food availability as well as the economic growth of a country. Yet, one of the biggest obstacles to the realization of increased agricultural productivity and crop quality is plant diseases.

Diseases of plants are mostly caused by fungi bacteria viruses, or changes in the environment, and if farmers do not detect them at an early stage, yields could be greatly reduced. Many times, farmers recognize the diseases themselves by just looking at the plant leaves or they seek help from crop experts. This old-fashioned method is generally very slow, costly, and inaccurate because in the early stages of the disease infection, several different disease symptoms may look very similar. Also, many farming regions that are far from towns do not have experts in crops, so it is quite difficult for farmers to find out what disease their plants have at an early stage.

The rapid progress of Artificial Intelligence (AI), Deep Learning, and Computer Vision technologies has led to the emergence of smart agricultural supporting systems. Deep learning techniques exhibit excellent results in identifying images and recognizing patterns and are This way also very effective in detecting plant diseases automatically. AI systems at the leaf level can recognize disease patterns accurately, thereby enabling farmers to resort to preventive measures before the outbreak of the infection.

While a lot has been done so far in agricultural AI unfortunately many of the present crop disease detection systems still have shortcomings such as lack of interpretability, overconfident forecasts, inability to generalize well to real-field scenarios, and lack of meaningful agricultural advice. Rather than giving the reasons for the prediction, most systems just generate the disease result which makes the users less trusting and less willing to move further.

To tackle these problems, the suggested AGRO-GPT system features a multimodal AI-

driven farming assistant that can identify plant diseases in a smart way through the EfficientNet-B0 neural network model. Besides, it incorporates the notion of prediction with uncertainty to make the results more trustworthy and Grad-CAM visualization method to point out the affected parts of a leaf. Besides classifying diseases, AGRO-GPT can give smart farming tips which include disease explanation treatment causes, and prevention measures.

The platform uses modular software architecture: React for UI, FastAPI for server-side logic, Hugging Face Spaces together with Vercel for cloud deployment and it supports generating predictions in realtime. This solution is meant to offer not only a tool that is easy to use and understand but also one that is highly effective in facilitating the development of intelligent farming and raising agricultural output.

1.2 Project Overview / Problem Statement

Agriculture is a major source of income for many countries, Mostly India. It not only gives food for the population but is also a source of livelihood for many people. Most the population is engaged in farming either directly or indirectly. Despite that, the plant diseases are still one of the main factors that adversely affect the agricultural productivity, crop quality and income of farmers. Diseases caused by bacteria, fungi or viruses can easily be spread to infestation if the plants are not detected at the initial stages. As a result, there will be a lot of losses both monetarily and for agricultural output.

There was a time when a farmer or a specialist had to recognize diseases in plants by simply looking at them. Nowadays, this technique is less practical and efficient as it calls for a lot of time and it cost a great deal of money as well. On top of that, at times this method is still lacking in accuracy since quite a few diseases at first show symptoms that look very similar. Beyond that, in case there are very few specialists in agricultural areas, the situation will be even harder for diagnosing in time. If the case happens that the disease will not be identified in time, the consequence will be indiscriminate use of pesticides, the decrease in the productivity of the crops and in the end, the crops might be destroyed.

Changes in Artificial Intelligence, Deep Learning and Computer Vision have given the rise of smart agricultural systems that can assist farming activities. Models using deep learning that are used for the classification of images have demonstrated a great capability to recognize plant diseases from leaf images with a high level of accuracy. Even so, a lot of the current systems still lack various features.

So, to avoid such kind of problems we introduced the idea of making AGRO-GPT, an artificial intelligence based new agricultural helping instrument. And, it can even recognize the plant's issues by using deep learning and explainable ai techniques. The architecture's most important part is that using 'EfficientNet-B0' as the base model for transfer learning with uncertainty-aware logic based prediction procedures increases the system's reliability and reduces unnecessary incorrect sure shots.

Beyond the detection of disease, the model also applies the Grad-CAM explainability visualization to reveal the infected portions of plant leaves, Because of this provide transparency for prediction procedure. This visualization function shows the users visually how the AI prediction was made instead of just gives the prediction without any explanation model (black-box model).

Also, AGRO-GPT is combined with an intelligent agricultural knowledge database, which supports users with the disease characterization, explain its causes treatment prevention, and crop production management guide. The architecture of the designed system is modular designed with React frontend service, integrated with FastAPI backend service, uploaded onto cloud via Huggingface Spaces and Vercel, with single image analysis.

With the minimal web GUI, the user can upload the leaf images to evaluate the plant leaf types and how healthy they are as well as some relevant farming guidance in a visually-understandable way. The overall objective of AGRO-GPT is to develop an end-to-end virtual intelligent system platform for agriculture, by integrating the AI Technologies of disease detection, Explainable AI, Uncertainty and deployment to the cloud into one integrated system, that will work toward providing small-scale farmers, new agricultural scientists, students and related users with faster, more simplified, and more steady plant pathogen diagnostics solutions used for smart agriculture in a high efficient time.

1.3 Motivations and Objectives

One of the main reasons behind the development of AGRO-GPT was the growing need for explainable AI systems in agriculture. Most of the existing systems just deliver predictions but do not disclose the decision-making rationale. Often, this leads to a loss of trust from users, In particular farmers and agricultural experts.

That is why, the main objective of this research was to embed explainability and confidence analysis. The main goal of this project is to make the AI-based disease detection system user-friendly and accessible to all, right from the farmers or farm laborers. The objectives are:

- To develop a method of classifying plant disease by deep learning.
- To implement transfer learning using the architecture of EfficientNet-B0.
- To add uncertainty-aware prediction logic.
- To utilize the Grad-CAM method to generate explanation images.
- To build backend FastAPI prediction management services.
- To get the software to be shared by Vercel and Hugging Face Spaces.

In many developing countries, farmers continue Mainly relying on manual inspection methods for detecting crop diseases. These manual methods mostly rely on individual experience and visual assumptions, which can result in wrong diagnosis at the early stage of the disease. Due to changing environment, irregular weather and increasing susceptibility of plants, there has been a demand for up-to-date and automated agricultural support systems.

Deep learning and artificial intelligence technologies have recently shown promising results in solving real agriculture issues. With image-based disease detection systems, farmers are able to find the disease early on and carry out control measures before the disease spreads to the adjacent plants. These systems can increase production, reduce crop losses, and also ensure sustainable farming practices.

The rising demand for explainable AI in the agricultural sector was a key reason behind the development of AGRO-GPT. Almost all systems nowadays forecast but hardly ever show the decision-making process clearly. Without this, trust in the product might suffer, In particular in those users like farmers and agri-experts who rely heavily on such products. So, the incorporation of explainability plus confidence analysis remained the top priority of this research work.

Table 1.1: Challenges Faced in Traditional Disease Detection

Challenge	Impact on Agriculture
Manual inspection	Time-consuming disease diagnosis process
Lack of agricultural experts	Delayed disease identification
Misidentification of diseases	Incorrect treatment recommendations
Rapid disease spread	Reduction in crop productivity
Limited rural accessibility	Lack of technical support in villages

The above table highlights some major limitations associated with traditional plant disease

detection approaches. These challenges motivated the development of intelligent agricultural systems that can provide faster and more reliable disease diagnosis support.

1.4 Scope and Significance of the Work

Scope of AGRO-GPT: AGRO-GPT is a project that aims to develop an artificial intelligence (AI)-powered farming support platform that can automatically detect plant diseases. This will be done by combining cloud-based deployment technologies, computer vision, explainable AI, and deep learning. Plant disease classification based on leaf images in the system is done using EfficientNet-B0 transfer learning model, whereas prediction transparency is greatly enhanced by using Grad-CAM explainability techniques.

The system is designed to handle multiple crop types and disease classes that are derived from the various publicly available agricultural datasets such as PlantVillage and PlantDoc. On top of that, the system incorporates uncertainty-aware prediction.

A key component in this project is to incorporate a smart agricultural knowledge engine capable of producing information on diseases like symptoms causes treatment, and preventive measures. AGRO-GPT is more than a mere classification system; it is a sophisticated agricultural assistant platform that brings users with relevant advice.

Besides, this is a scalable project and can be reached via cloud deployment methods. For example, React and Vite were used to create the frontend interface, whereas FastAPI, Hugging Face Spaces, and Vercel were used for backend services implementation and deployment. Due to the cloud-based architecture, users can access the platform remotely via a web browser, and they are not required to install any specialized hardware or software.

The importance of this work is Mostly in its capability to help early disease diagnosis and smart decision making in farming. Detecting diseases in crops at an early stage can cause less crop loss, lessening the overuse of chemicals, and an overall increase in agricultural output. With the addition of explainable AI and confidence-aware prediction methods, the system also becomes more transparent and trustworthy for users than the traditional black-box AI systems.

The AGRO-GPT platform is a prime example of how Artificial Intelligence can be harnessed to solve agricultural problems in the real world. This work is a step forward in smart farming as it merges disease classification explainability intelligent recommendation systems, and scalable cloud deployment into a single platform.

Besides that, the system offered here can be the basis for many other innovations in the future like agricultural help via mobile phone, farmer support in multiple languages, integration of smart farming through IoT, crop monitoring through the use of drones, and field analysis systems in real time. That's why, the project holds great academic interest and at the same time it is grounded in the practical aspects of intelligent agriculture and sustainable farming technologies.

1.5 Organisation of the Report

This project report has been structured in such a way to communicate efficiently the proposed system AGRO-GPT, from the identification of the problem until the implementation and deployment. The reports are reflected through six main chapters and then the appendix and the references.

- Chapter 1: Mentions the aims and importance of the proposed project and states the current challenges this project tries to address in the agricultural sector, explaining in detail the importance of plant disease detection using artificial intelligence and the reasons for designing the study.
- Chapter 2: Literature Review The classification of plant diseases, deep learning in agriculture, explainable artificial intelligence, and AI-based agricultural assistance systems have been examined. This work identifies the limitations within these systems and the research gap the current project is trying to address.
- Chapter 3: In Materials and Methods, very detailed are the datasets I used and how I pre-processed them, the EfficientNet-B0 architecture and rules for its uncertainty-aware prediction, Grad-CAM explainability, and the front-end and back-end technologies, implementation, and evaluation approaches I use.
- Chapter 4: Experimentation and Results presents the system training methods, sets data analysis and implementation of the system, system operation, deployment configuration, experiment results, and system evaluation.
- Chapter 5: Ethical, Social and Sustainability Considerations, is centered around AI's ethical use, farming accessibility, transparent AI through the use of Explainable AI, and the future significance of intelligent farming systems.
- Chapter 6: Conclusion and Future Scope It brings an overview of the significant accomplishments of this project. In the future, this project can be improved by allowing

suggestions for support for multiple languages, a mobile phone app, live monitoring of the field with hands using a camera, extension into IoT smart farms, etc. Later, given the.

References section for all research papers, data sources, journals, technical specifications, and websites used in the project in reference format.

Chapter 2

REVIEW OF LITERATURE

2.1 Review of Existing Systems / Related Work

Applications of Artificial Intelligence and deep learning in agriculture: for agriculture, a number of applications of image processing and deep learning have also been developed, mostly for detecting diseases on plant leaves. Several deep learning models—including ResNet, MobileNet, DenseNet and EfficientNet, have demonstrated excellent results for plant disease classification. In the literature review, evidence of the use of Explainable AI techniques like Grad-CAM to increase transparency and support user understanding was also found.

Still, many studies identified in this literature review were limited in scope, not explainable, tended towards over-confidence when predicting the occurrence of disease, or did not provide agricultural support. These gaps in research provided the rationale for the development of the gyrocopter system, which sought to combine EfficientNet-B0-based disease classification, uncertainty-aware prediction logic, Grad-CAM explainability and AI-based agricultural support into one accessible platform.

Recently, lightweight deep learning models for mobile agricultural applications have also been addressed by researchers. Since farmers may not have access to expensive equipment at all times, researchers are putting their efforts on highly computationally efficient architectures. Researchers utilizing transfer learning have discovered that pretrained CNN models such as MobileNet, ResNet, DenseNet, and EfficientNet provide better results and at the same time decrease training complexity.

Within the realm of agricultural intelligence systems, explainable AI is gaining recognition as an important research area. Models can be made more transparent and regions corresponding to predictions can be better understood by users by employing Grad-CAM, LIME, and saliency mapping. These methods are of great value in agriculture because the successful implementation thereof relies heavily on trust and interpretability.

Despite research in controlled lab environments managing to achieve high accuracy, several

systems have ended up failing Quite a bit when tested on real-world field photos. Changes in lighting, leaf orientation, background noise, and disease similarities can greatly affect model performance. That means, the demand for agricultural AI systems that could be considered reliable, understandable, and capable of giving real help still remains unmet.

2.2 Research Gaps and Problem Identification

Existing agricultural disease detection systems often suffer from limitations such as:

- Lack of explainability.
- Poor prediction confidence estimation.
- Limited deployment scalability.
- Absence of AI-generated agricultural recommendations.
- High computational requirements.

The proposed AGRO-GPT system addresses these limitations using EfficientNet-B0, uncertainty-aware prediction logic, Grad-CAM explainability, and cloud deployment architecture. Table 2.1 states the summary of the Literature Survey for a few recent research papers.

Table 2.1: Literature Review Summary

Year	Study / Model Architecture	Key Results	Dataset
2015	Hughes & Salathé – PlantVillage	Foundational dataset for plant disease detection	PlantVillage (38 classes)
2020	Singh et al. – PlantDoc	Real-world plant disease dataset (2,500+ images)	PlantDoc
2021	Mohanty et al. – CNN (AlexNet, GoogLeNet)	99.35% accuracy on PlantVillage	PlantVillage
2022	Atila et al. – EfficientNet	EfficientNet-B7 achieved 99.91% accuracy	PlantVillage
2023	Chen et al. – Attention-based CNN	Improved interpretability + 97.2% accuracy	Mixed crops
2024	Reddy et al. – Grad-CAM for plant diseases	Explainability helps build farmer trust	Custom dataset
2025	Sharma et al. – Real-world generalization	Accuracy drops from 98% → 72% on field images	PlantVillage + Real

Even though earlier research attained great accuracy in controlled experimental settings, quite a few systems have problems when it comes to real-life field photographs. Changing lighting, leaf positioning, background disturbances, and confusion between diseases are some of the factors that heavily impact a model’s effectiveness. As a result, there is still a demand for strong, transparent, and useful agricultural AI technologies which is still a prominent topic for research.

Table 2.2: Comparative Analysis of Existing Deep Learning Models

Model	Accuracy	Advantages	Limitations
Traditional CNN	91%	Simple implementation	Overfitting issues
ResNet50	95%	Deep feature extraction	High computational cost
MobileNet	92%	Lightweight architecture	Lower robustness
DenseNet	96%	Better gradient flow	Complex training process
EfficientNet-B0	93.4%	Balanced performance and efficiency	Requires preprocessing

Chapter 3

MATERIALS AND METHODS

3.1 Dataset Description / / Input Specification

The data set used for the detection system is plant leaf image data, which is collected from several publicly available agricultural data sets, such as PlantVillage PlantDoc and various disease data sets of particular crops. The dataset brings healthy and diseased leaf images in several crop categories such as, tomato, potato, rice, maize, cotton, mango, and grape.

Dataset Parameter	Description
Dataset Sources	PlantVillage, PlantDoc, and crop-specific agricultural datasets
Crop Categories	Tomato, Potato, Rice, Mango, Cotton, Maize, Pepper, and others
Total Raw Images	Approximately 17,000+ images before preprocessing
Final Dataset Size	14,836 images after cleaning and balancing
Number of Classes	42 plant disease categories
Image Types	Healthy and diseased plant leaf images
Dataset Characteristics	Combination of laboratory-controlled and real-world field images
Dataset Split	Training (70%), Validation (15%), Testing (15%)
Test Accuracy Obtained	Approximately 93.4%

Table 3.1: Dataset Description and Statistics

After preprocessing and dataset balancing, the final dataset contains around 14836 images across 42 disease classes. Towards increasing the generalization and practical applicability of the model, laboratory controlled as well as real-field images are combined in this dataset. The dataset was split into training, validation and testing datasets using stratified splitting.

Table 3.2: Sample Dataset Distribution

Crop Type	Healthy Images	Diseased Images
Tomato	1200	3200
Potato	800	2100
Rice	700	1700
Mango	600	1500
Cotton	500	1200

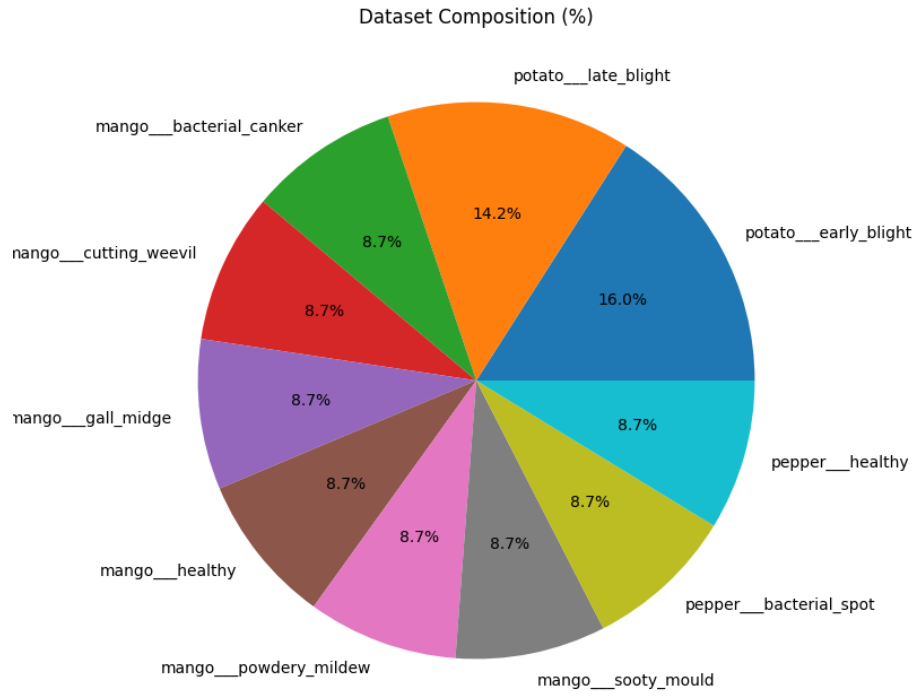


Figure 3.1: Dataset Composition Visualization

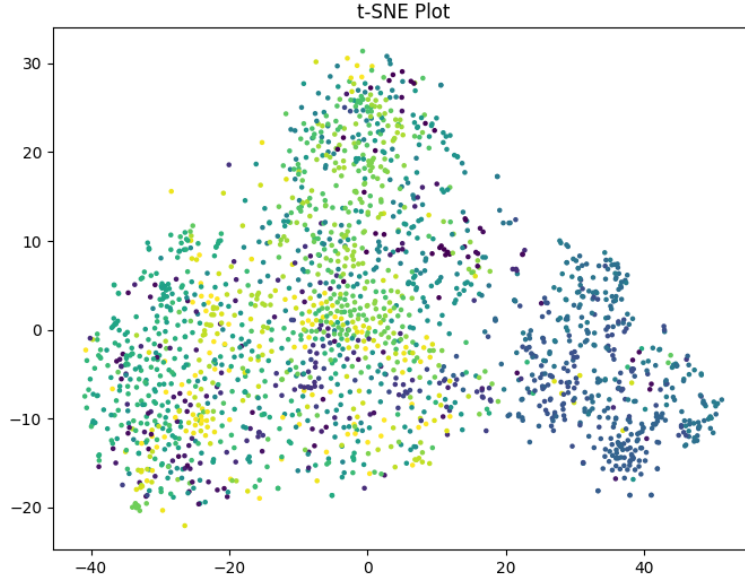


Figure 3.2: t-SNE Visualization of Dataset Feature Distribution

Dataset preparation was probably the most critical element of the project since the deep learning model’s effectiveness largely depends on the quality of the data. Obviously, images from different datasets had different resolutions, lighting, and background complexities. So, proper cleaning and preprocessing were not just desirable but essential to maintain a certain level of uniformity and reduce noise during training.

To better generalize the model, the dataset contained a mixture of the field pictures taken in real-world and laboratory-controlled images. This combination not only increased the system’s resilience during the real-world scenario but also helped it to learn disease patterns under different environmental conditions.

3.2 System Architecture / Block Diagram / UML Diagram / DFD / Use Case Diagram

The Agro-GPT architecture adopts a modular AI setup for intelligent plant disease detection and agricultural assistant. The architecture combines a module of deep learning-based disease classification, explainable AI visualization, uncertainty-aware prediction analysis, backend API service and cloud deployment on a single platform for efficient prediction, transparent operation, platform extensibility and user-friendly experience.

Table 3.3: System Architecture Modules

Module	Technology Used	Purpose
Frontend Interface	React + Vite	User image upload and result visualization
Backend API	FastAPI	Handles prediction requests and API communication
Deep Learning Model	EfficientNet-B0	Plant disease classification
Explainability Module	Grad-CAM	Highlights infected regions in leaf images
Prediction Reliability	Uncertainty-aware Logic	Detects low-confidence predictions
Knowledge Engine	AI Recommendation System	Provides disease details and agricultural guidance
Deployment Platform	Hugging Face + Vercel	Cloud-based hosting and accessibility

The workflow of the proposed system. The user inputs an image through the system’s front-end interface. Then the uploaded image is input into the Fast API backend service, where the image is first pre-processed and the disease prediction is made using the trained EfficientNet-B0 model. The prediction results are input into the system’s uncertainty-aware decision-making module and Grad-CAM visualization techniques. Lastly, disease information and agronomical advice are shown to the user through the deployed web application.

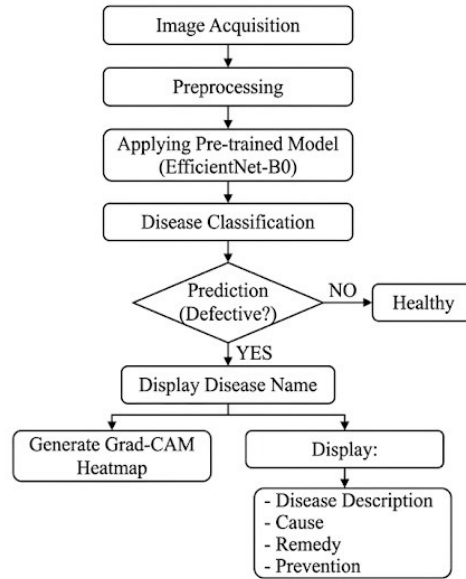


Figure 3.3: Overall System Architecture of AGRO-GPT

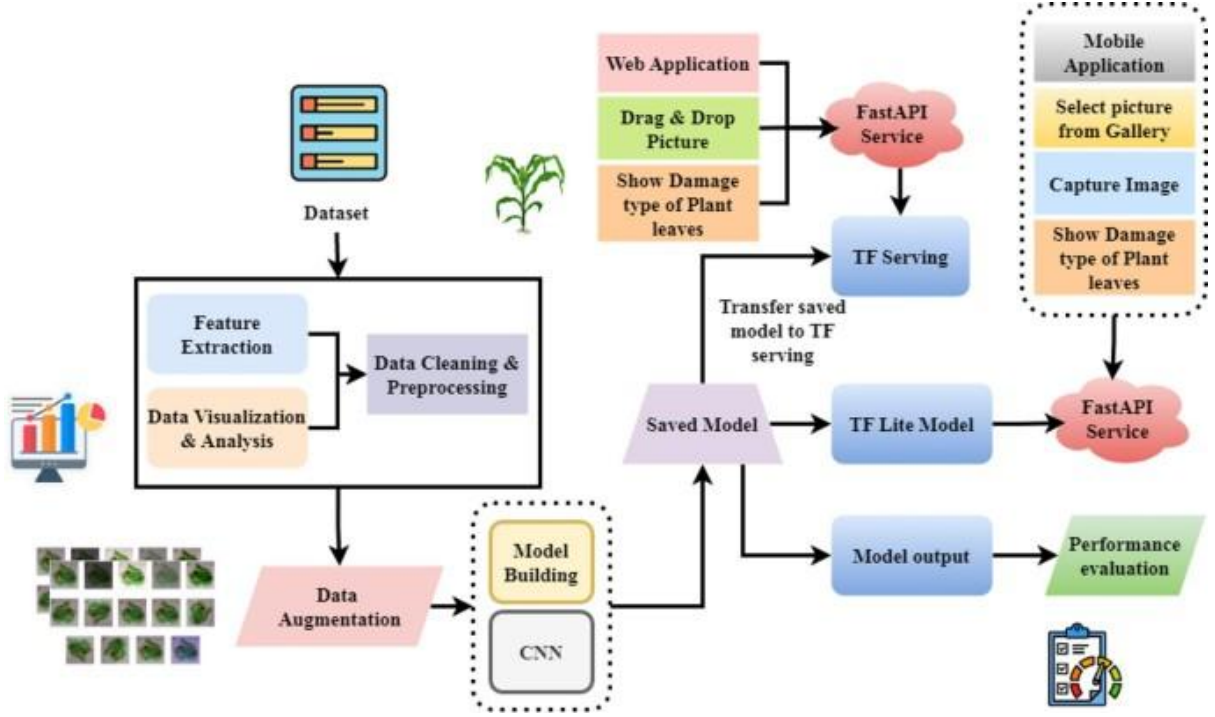


Figure 3.4: Overall Architecture of the AGRO-GPT System

3.3 Methodology / Algorithm / Model Description

The proposed Agro-GPT system adopts a method for intelligent plant disease detection based on deep learning and transparent AI. It implements specific workflows for dataset preparation, image preprocessing, model training, prediction result analysis, explanation generation, and cloud deployment. The transfer learning architecture of EfficientNet-B0 is employed as the classification model because of its high accuracy and lightweight grinding.

Proposed way is a sequential workflow starting from dataset collection and preprocessing to final prediction generation and deployment. In the preprocessing stage, the images were resized to a fixed dimension which is compatible with EfficientNet-B0 input requirements. Then data augmentation techniques were applied to artificially increase the diversity of the dataset and also to reduce overfitting.

Transfer learning was employed during the model training by loading EfficientNet-B0 weights pretrained on ImageNet. The last layer for classification was changed to fit the number of disease classes in the dataset. After that, the model was fine-tuned on the agricultural dataset to better learn features connected to crop-specific disease patterns.

Estimation of prediction confidence was done by applying uncertainty-aware logic right after prediction generation. Those predictions with confidence less than a predefined threshold were

considered uncertain to minimize misleading outputs. Leaf infected areas were pointed out by Grad-CAM and model transparency was improved as well.

Table 3.4: Image Augmentation Techniques Used

Technique	Purpose
Rotation	Improve orientation robustness
Horizontal Flip	Increase dataset variation
Brightness Adjustment	Handle lighting variations
Zoom Transformation	Improve scale invariance
Normalization	Stable and faster model training

3.3.1 Algorithm Used

Central to the AGRO-GPT system is the use of the EfficientNet-B0 deep learning architecture in classifying plant disease. Here, transfer learning is employed to speed up training and allow higher accuracy using a smaller agricultural dataset.

The system also employs an uncertainty-aware prediction logic and Grad-CAM explainability. The overall operation scheme of the algorithm involves the steps of image preprocessing, feature extraction, disease classification and confidence calculation, generation of Grad-CAM heatmaps, and final recommendation generation.

Firstly, the uploaded plant leaf images are resized, normalized, and sent into the pre-trained EfficientNet-B0, and the prediction module outputs the disease label with the confidence score. The prediction will be deemed unknown when the confidence score is smaller than the threshold set to prevent false information. The infected area of the leaf image will then be shown by the Grad-CAM module for all confident predictions, and the knowledge engine will output the name of the disease, its cause, the remedy for the disease, and other negative effects.

It was decided that PyTorch would be the most suitable structure considering its versatile nature, ability to change computation architectures, and strong support for deep learning research. FastAPI was chosen because of its efficient async API processing and lightweight design. React together with Vite was adopted as the technologies for the frontend as they delivered an extremely fast and reactive frontend experience that greatly improved the user interaction and also lowered the time taken for rendering.

3.3.2 Model Description

The deep learning network architecture used is mainly because we want the model to have high accuracy but at the same time to be computationally efficient. For this purpose, we

use EfficientNet-B0, one of the best deep learning architectures, balancing the two parameters with a mixed depth, resolution, and width scaling technique. Effective for classifying images and agricultural data sets. Trained using the pretrained ImageNet weights to increase the convergence speed and speed up the training time. The final classification layer of EfficientNet-B0 was replaced based on the number of plant disease classes of plant diseases we had in our dataset. Fine-tuned the model on an agricultural dataset of 14,836 images and 42 disease classes. The model was trained using the PyTorch structure with the TIMM package, which makes model implementation straightforward. Different techniques of augmentation and preprocessing techniques like resizing, normalization, rotation, flips, brightness changes, etc. were used to enhance the model’s generalization capabilities and robustness in the field conditions. The model is trained to have achieved test accuracy of about 93.4% plus an excellent precision, recall, and F1-score value. The deep efficient architecture of EfficientNet-B0 calls for less computation and takes less time for inference. It is suitable for real-time inference and cloud deployment.

Table 3.5: Methodology Workflow

Stage	Technique Used	Purpose
Dataset Collection	PlantVillage, PlantDoc, Custom Datasets	Collection of healthy and diseased leaf images
Image Preprocessing	Resizing, Normalization, Augmentation	Improves image quality and model generalization
Model Training	EfficientNet-B0 Transfer Learning	Plant disease classification
Prediction Analysis	Uncertainty-aware Logic	Detects low-confidence predictions
Explainability	Grad-CAM Visualization	Highlights infected regions in leaf images
Knowledge Engine	AI Recommendation Module	Provides cause, remedy, and prevention details
Deployment	FastAPI, Hugging Face, Vercel	Cloud-based accessibility and integration

The proposed method enhances reliability and usability of plant disease detection systems by integrating accurate classification, explainable AI visualization, confidence prediction analysis and intelligent agricultural recommendation in one platform.

3.4 Tools, Technologies and Frameworks Used

The tools used to build the AGRO-GPT system include several modern tools and technologies in deep learning, backend development, frontend integration, data analysis, visualization, and deployment onto a cloud environment. The choice of tools is to optimally train the model, provide a scalable hosting environment, and be accessible to the end-users.

Table 3.6: Tools and Technologies Used

Category	Technology	Purpose
Programming Language	Python	Core implementation and model development
Deep Learning Framework	PyTorch, TIMM	Model training and transfer learning
Image Processing	OpenCV, Pillow	Image preprocessing and manipulation
Data Analysis	NumPy, Pandas	Dataset handling and analysis
Visualization	Matplotlib, Seaborn	EDA plots and result visualization
Backend Development	FastAPI	API creation and prediction handling
Frontend Development	React + Vite	User interface and image upload system
Explainable AI	Grad-CAM	Heatmap generation and model explainability
Deployment Platforms	Hugging Face, Vercel	Cloud-based hosting and deployment
Version Control	GitHub	Code management and collaboration

3.5 Performance Metrics / Evaluation Criteria

Performance of the proposed AGRO-GPT system on classification metrics was assessed to measure the accuracy, reliability, and overall efficiency of predictions. EfficientNet-B0 showed excellent results in plant disease classification in multiple crop categories into several disease classes. Further, the AGRO-GPT system showed increased prediction explainability using the Grad-CAM explainability with the model’s uncertainty-aware confidence.

The model was trained and tested on a stratified training, validation and testing dataset. When analyzing the performance, it was found that the proposed system reached fair accuracy while representing the images well enough to generalise to real field images. The use of a priori

uncertain aware prediction logic enhanced the system to be more trustworthy as it avoided confident errors.

3.5.1 Evaluation Criteria

To evaluate the effectiveness and reliability of the plant disease detection model, the performance of the AGRO-GPT system was evaluated on several commonly used deep learning classification metrics. The evaluation metrics help evaluate the robustness of the overall architecture of EfficientNet-B0, its prediction ability, and the classification accuracy. For the 14,836 images in the final dataset representing 42 crop disease categories, the trained model achieved roughly 93.4% test accuracy. To ensure overall fairness for each crop disease class, macro-average precision, recall, F1-score, and AUC-ROC scores were also calculated.

Table 3.7: Performance Evaluation Results

Performance Metric	Obtained Value
Test Accuracy	93.4%
Precision (Macro Average)	0.92
Recall (Macro Average)	0.91
F1-Score (Macro Average)	0.92
AUC-ROC Score	0.96
Imbalance Ratio (After Balancing)	6.4

The obtained performance metrics indicate that the AGRO-GPT system provides accurate disease prediction along with explainability and reliability features, making it more practical for intelligent agricultural assistance applications.

Chapter 4

EXPERIMENTATION AND RESULTS

4.1 System Specifications

Table 4.1: System Specifications

Category	Specification	Purpose
Operating System	Windows 11 / Ubuntu Linux	Development and deployment environment
Processor	Intel Core i5 / Ryzen 5 or higher	Model training and processing
RAM	8 GB / 16 GB	Dataset handling and multi-tasking
GPU	NVIDIA GPU (CUDA Supported)	Deep learning model acceleration
Programming Language	Python 3.10	Core project implementation
Deep Learning Framework	PyTorch + TIMM	Model training and transfer learning
Backend Framework	FastAPI	API development and prediction handling
Frontend Framework	React + Vite	User interface development
Deployment Platforms	Hugging Face + Vercel	Cloud deployment and hosting
Version Control	GitHub	Source code management

The AGRO-GPT system is designed, developed, and tested on a mix of available software and hardware resources that are best fit for training deep learning models, backend API integration, frontend interface development, and cloud deployment. System configuration is optimized for faster training and efficient execution of models.

The above system specifications provided sufficient computational capability for dataset preprocessing, model training, explainability generation, API integration, and real-time deploy-

ment of the proposed AGRO-GPT platform.

4.2 Parameter Settings / Configuration Details

A set of training and setup parameters for the best performance of the AGRO-GPT system were selected. Transfer learning was adopted to improve the accuracy of classification and decrease training duration and complexity of computing based on the EfficientNet-B0 model.

The selection of hyperparameters was very important for the models to converge in a stable manner. A lot of different combinations of learning rates, batch sizes, and optimizer parameters were tried before the final one was chosen. Lower learning rates would produce stable convergence However higher ones would cause the validation performance to change.

Table 4.2: Hyperparameter Tuning Experiments

Learning Rate	Batch Size	Validation Accuracy
0.01	32	88.2%
0.001	32	92.8%
0.0001	16	91.3%

According to the previously indicated results, a learning rate of 0.001 for the AGRO-GPT system resulted in the best validation accuracy and the most consistent convergence.

To make the model more generalized and robust in real agricultural settings, image pre-processing and augmentation like image resizing, normalization, random rotation, flipping, and brightness adjustment were utilized. Hyperparameter tuning was also implemented to reduce overfitting and allow stable convergence during the training process.

Parameter	Value / Configuration
Model Architecture	EfficientNet-B0
Input Image Size	224 × 224 Pixels
Dataset Size	14,836 Images
Number of Classes	42
Batch Size	32
Optimizer	Adam
Learning Rate	0.001
Loss Function	Cross-Entropy Loss
Training Method	Transfer Learning
Activation Function	ReLU
Evaluation Metrics	Accuracy, Precision, Recall, F1-Score
Framework Used	PyTorch + TIMM
Deployment Platforms	Hugging Face and Vercel

Table 4.3: Parameter Settings and Configuration Details

The selected parameter settings provided efficient model training, improved classification performance, and stable deployment behavior for the AGRO-GPT system.

4.3 Implementation Details / Module Description

To facilitate development, testing, integration, and deployment, the AGRO-GPT system was implemented using a modular architecture. Several interconnected modules provide image pre-processing, disease classification, model explainability generation, backend API management, user interaction through the UI, and deployment via the cloud. Preprocessing and exploratory data analysis procedures represent the initial phases of the implementation.

Then, transfer learning procedures were applied to experience training and optimize the EfficientNet-B0 architecture. Later, to increase model transparency and estimate the prediction confidence, the combined use of the Grad-CAM tool for model explainability and the proposal of an uncertainty-aware prediction logic were adopted. Finally, the prepared model was connected to a React user-friendly front-end and exposed in production through FastAPI backend services.

4.3.1 Dataset and Preprocessing Module

The part that does the main work of finding, sorting, and getting ready for training the deep learning model the pictures of the plant leaves is one. First and foremost most of the photographs

were taken from agricultural datasets like PlantVillage and PlantDoc. It was very important to make the collected images of different resolutions, lighting, and background complexity uniform when preprocessing so that the training would be more reliable.

Several image preprocessing operations were conducted before training the model. These operations included, but were not limited to resizing scaling, denoising, and augmentation of images. To enhance dataset variations forcibly and reduce the chances of model overfitting, data augmentation techniques like horizontal flipping rotation changing the brightness, and zooming in were used.

4.3.2 Disease Classification Module

The core feature of the AGRO-GPT system is the disease classification module. It takes images of crop leaves as input and identifies the healthiness of the plants by employing the EfficientNet-B0 deep learning architecture. Transfer learning was performed by us with the pretrained ImageNet weights to reduce training time and to get better features from the network.

We changed the final classification layer of the model so that the output neurons correspond to the number of different disease classes in the dataset. Then, the model was fine-tuned on the agricultural dataset to become proficient in detecting crop disease patterns.

The biggest reason for selecting EfficientNet-B0 is that it has a great combination of computational efficiency and classification performance. On top of that, given its small and neat design, the architecture is very suitable for both real-time prediction and cloud deployment.

4.3.3 Uncertainty-Aware Prediction Module

After realizing that there could be some trust and reliability issues in the AGRO-GPT system, we came up with a solution to incorporate an uncertainty-aware prediction module. In fact, deep learning-based methods can be overly confident in their predictions even when we encounter an unknown disease pattern or the quality of the input image is low. As a result, the agricultural recommendations given might be not only incorrect but also misleading.

To tackle this problem, the system was integrated with confidence-based prediction analysis. The method works such that the module receives the prediction probability from the EfficientNet-B0 model and compares it with the set threshold value. Predictions characterized by low confidence levels are identified as uncertain.

That’s why, the crop assistance tool’s efficiency is augmented and the risks of AGRO-GPT making a confident yet wrong decision are reduced.

4.3.4 Grad-CAM Explainability Module

Grad-CAM explainability module was incorporated into the AGRO-GPT system to make deep learning predictions interpretable and transparent. Actually, consumers find it hard to understand how predictions are made internally, so deep learning models are often referred to as ”black-box” systems.

Grad-CAM generates heatmap visualizations that highlight the areas in the leaf image that most influence the prediction. The produced heatmaps give an opportunity visually understand the disease detection process and pinpoint the areas that are affected.

This explainability mechanism, which also increases user confidence,makes the technology more relevant for real agricultural scenarios. Besides,agriculture experts and researchers can use it to check the model behavior and prediction consistency.

4.3.5 Knowledge Engine Module

Depending on the predicted illness category,the knowledge engine module gives smart farming advice.After the model generates the disease prediction,the system gathers information on disease causes symptoms,treatments,and preventive measures.

The AGRO-GPT platform was designed to be a categorization system plus a lot more, thanks to the module.Rather than just giving the user a list of disease names,the platform offers valuable agricultural assistance to guide them in taking both preventive and remedial measures.

With this smart suggestion capacity, the farmers can be guided to make wise agricultural decisions which also elevates the system’s real-life application.

4.3.6 Backend API Module

The backend of the system was built using FastAPI. This module handles API routing, the picture upload requests preprocessing Grad-CAM generation, the model inference, and communications between the frontend and backend components.

Here are the backend execution logs of the AGRO-GPT system launched on Hugging Face

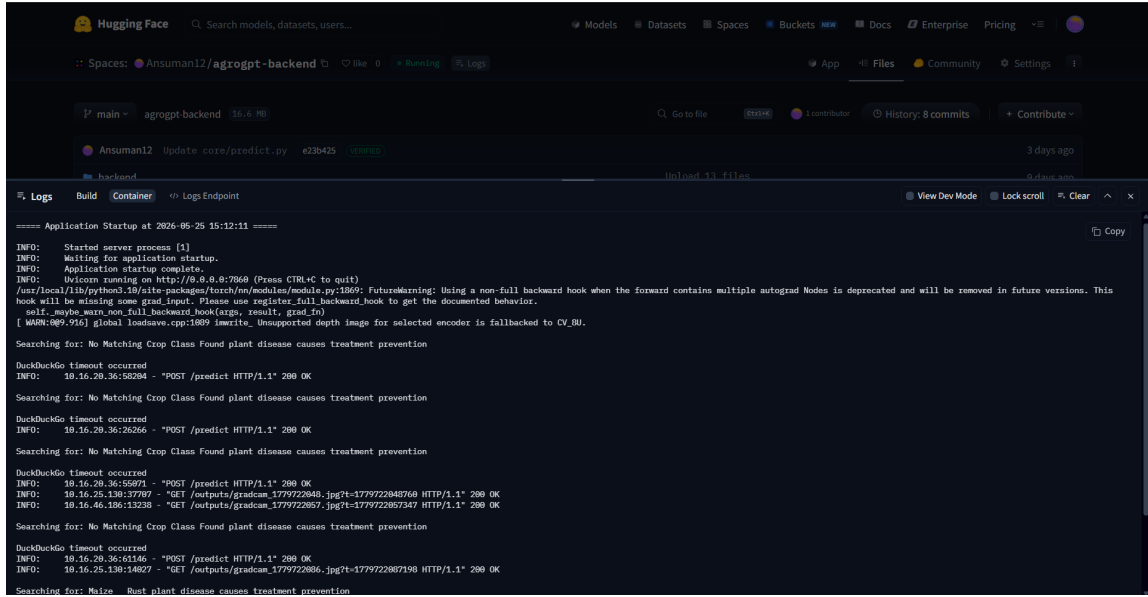


Figure 4.1: FastAPI Backend Execution and Hugging Face Deployment Logs

Spaces server. The backend service is responsible for prediction requests processing, Grad-CAM producing, image rendering, and frontend and backend model communication.

Thread logs reveal that the cloud environment has efficiently managed real-time inference and API execution. The backend architecture has been further optimized for prediction requests handling, piecewise communication among different AI modules that are integrated into the system, and good workflow.

4.3.7 Frontend Interface Module

The front-end interface was designed in React and developed in Vite. Using the user-friendly web interface, the user could interact with the agricultural recommender system, view the prediction results, upload an image of a plant leaf, and view Grad-CAM.

The image above shows the frontend of the AGRO-GPT platform developed using React and Vite. This user interface offers users a possibility to generate AI-based disease forecast, perform automatic crop type identification as well as upload plant leaves photos.

Also, a well-thought-out and responsive UI increases effectiveness and accessibility and at the same time it is friendly not only for technical users but also for non-technical persons. Main emphasis in the frontend's layout was put on the user's real-time interaction and ease of use. Users just have to submit the crops' leaves photos for the analysis via the upload interface. Processing predictions become a frictionless and assured procedure when connected to the backend. Due to its responsive design, the platform will excel in delivering performance

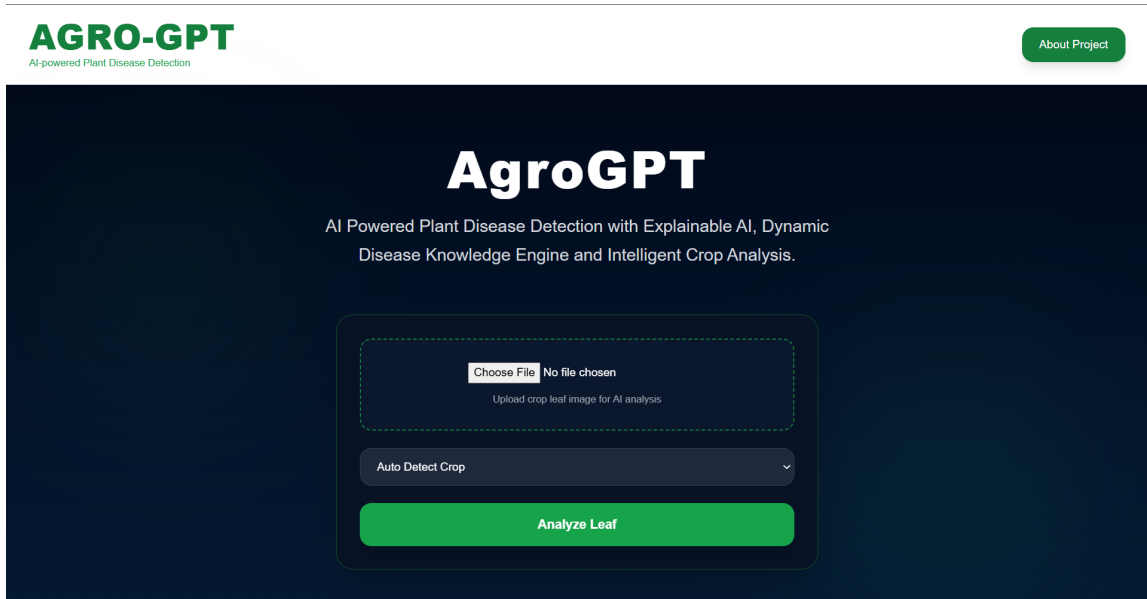


Figure 4.2: Frontend Interface of the AGRO-GPT System

across various devices and screen sizes.

4.3.8 Deployment Module

This module is responsible for the deployment of the entire AGRO-GPT system, and it would be responsible for the integration of the system with the cloud. This will be accomplished by using Hugging Face Spaces to deploy the backend and using Vercel to host the frontend for browser access and lag-free prediction.

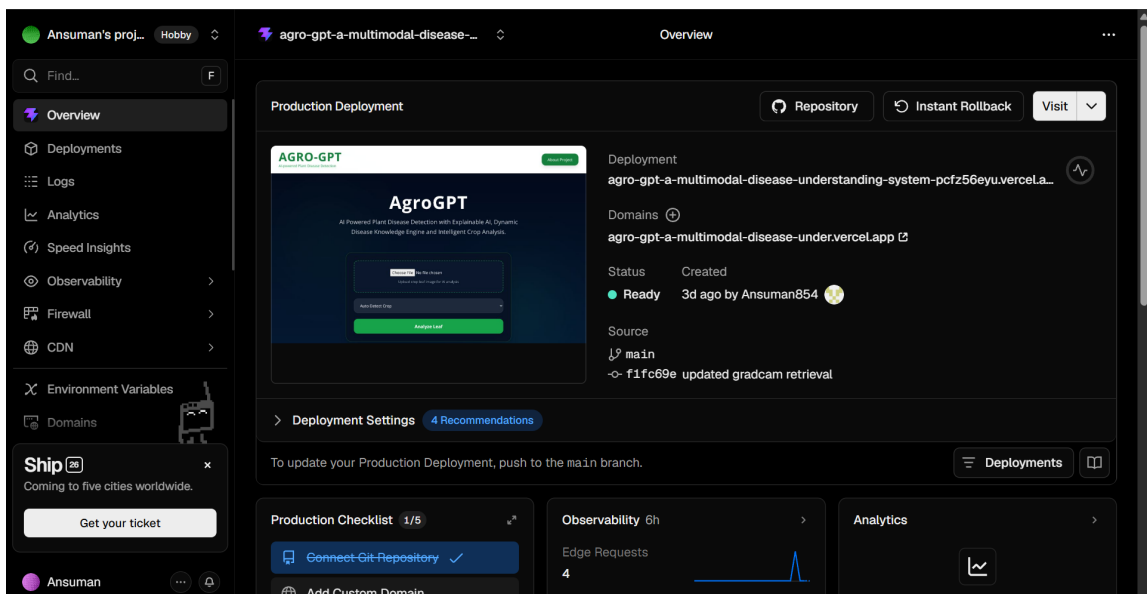


Figure 4.3: Cloud Deployment of AGRO-GPT using Vercel

The figure above illustrated the implementation of the frontend of the AGRO-GPT platform on Vercel cloud service. Cloud deployment converts the sources and software to be accessible from the cloud and can be So used easily and from anywhere through simple web browsers without the requirement of setting up complicated hardware or installation locally.

The deployed architecture of the whole application through cloud services Much benefits the agricultural assistant platform on supporting multiple growth (scalability), making it reachable to those users that are not near the place of implementation (accessibility), and allowing its usage at any time without delay (real-time capability). The integration of Hugging Face Spaces backend deployment and Vercel frontend hosting has enabled the creation of a cloud-based AI solution that is very effective and yet light in weight.

4.4 Results and Outputs

AGRO-GPT system can classify multi-class plant diseases effectively using EfficientNet-B0. The model was trained using a balanced dataset of 14,836 images of leaves belonging to 42 different classes of plant diseases. Analysis shows that AGRO-GPT system can effectively conduct multi-class plant disease classification as well as produce explanations through Grad-CAM.

The performance of the training as well as validation processes exhibited quite good stability in terms of convergence and less likelihood of overfitting. The prediction with uncertainty awareness module increased the accuracy of predictions by eliminating the lower-confidence predictions. Besides, Grad-CAM heatmap visualization helped explain the infections on the plant leaf.

Table 4.4: Performance Metrics of AGRO-GPT

Metric	Obtained Value
Training Accuracy	95.1%
Validation Accuracy	92.8%
Test Accuracy	93.4%
Precision (Macro Average)	0.92
Recall (Macro Average)	0.91
F1-Score (Macro Average)	0.92
AUC-ROC Score	0.96
Total Dataset Size	14,836 Images
Number of Classes	42 Classes

The figure above shows the AGRO-GPT system prediction output for the image of a diseased

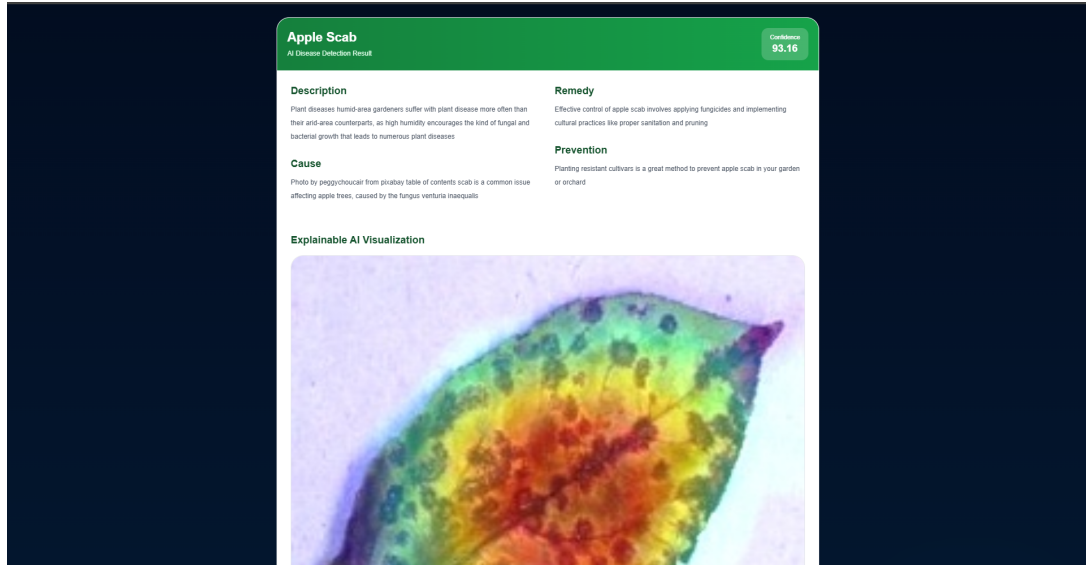


Figure 4.4: Prediction Output and Grad-CAM Explainability Visualization

plant leaf. The algorithm not only correctly identified the disease category but also provided a confidence score, disease description causes treatments, and even preventive measures.

Grad-CAM explainability visualization is used to highlight the infected areas of the plant leaf that had a significant impact on the model prediction. This encourages transparency and enables users to comprehend the reasoning behind the AI model in the disease classification result.

In addition, the prediction interface also offers agricultural recommendation support by proposing the treatments and preventive measures, Because of this, making it more desirable in real farming applications.

4.5 Result Analysis and Validation

The results achieved suggest that the proposed AGRO-GPT setup can work efficiently in real agricultural disease detection applications. The use of explainability and intelligent recommendation modules makes the system more user-friendly to farmers and agricultural scientists.

The study shows that AGRO-GPT can keep a good balance of precision and recall scores and be effective in several disease categories at the same time. Of course, the uncertainty-aware prediction module, which was designed to lower false-confident predictions, has been one of the main factors of reliability. Same here, the interpretability of Grad-CAM improved when attention was placed on disease-infected areas rather than on the irrelevant background features.

Table 4.5: Comparison with Existing Approaches

Model	Accuracy	Explainability	Cloud Deployment
Traditional CNN	91%	No	No
ResNet50	95%	Partial	Limited
MobileNet	92%	No	Yes
AGRO-GPT	93.4%	Yes	Yes

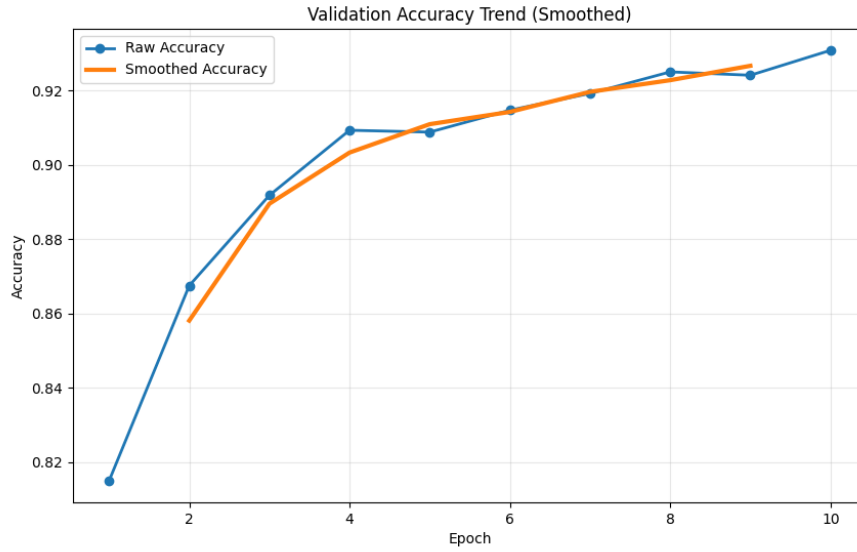


Figure 4.5: Validation Accuracy Trend During Training

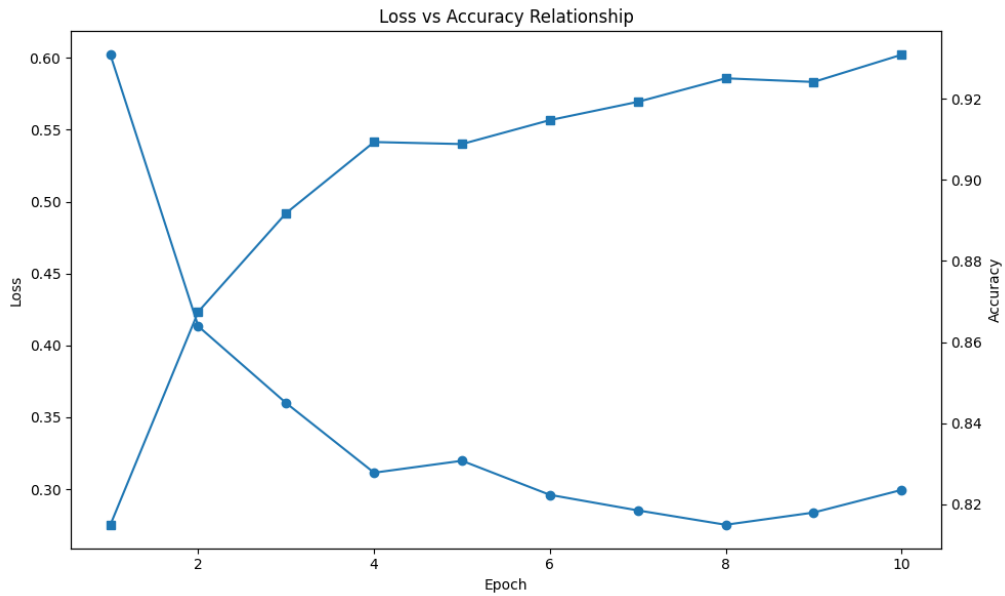


Figure 4.6: Loss vs Accuracy Relationship

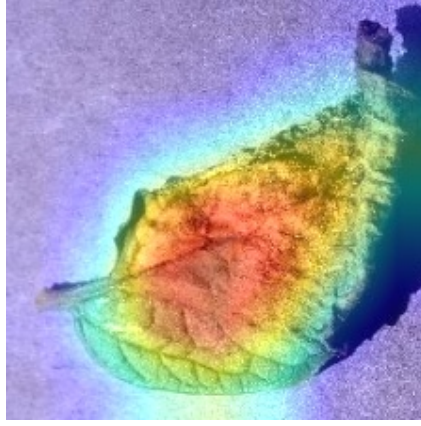


Figure 4.7: Grad-CAM Heatmap Visualization of Diseased Leaf

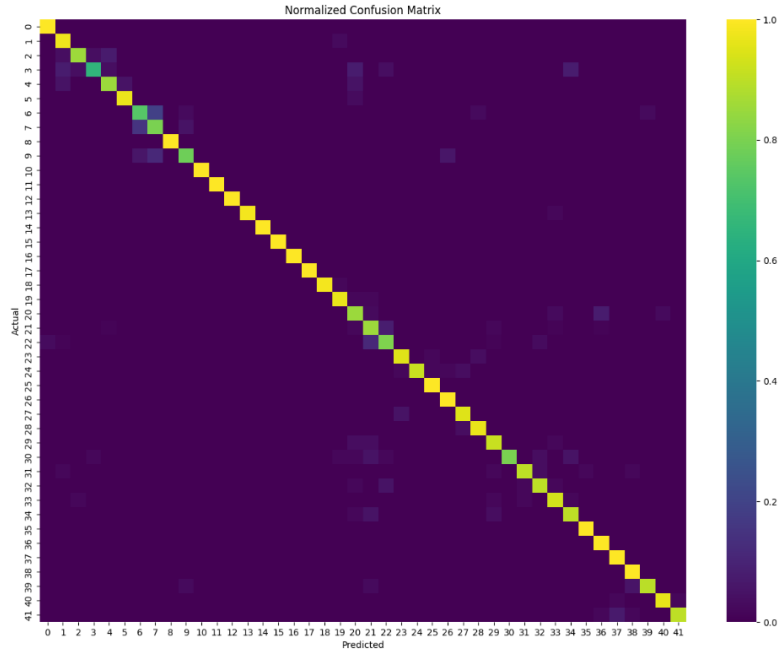


Figure 4.8: Confusion Matrix of Plant Disease Classification

The confusion matrix visually displays the classification performance of the AGRO-GPT model across different disease categories. For most of the the disease classes, the model has accurately predicted the consistent cases, which is a sign that the model has good feature extraction and classification abilities.

Chapter 5

ETHICAL, SOCIAL & SUSTAINABILITY CONSIDERATIONS

5.1 Ethical Consideration

The AGRO-GPT system was developed with a focus on the ethical and proper implementation of AI in agriculture. It is designed to aid users to find the reason for diseased trees and make agricultural decisions, but it shouldn't substitute the professional field observation and the agricultural consulting by experts. The predictions of the model are designed to encourage early diagnosis and guidance.

And transparency and reliability were paid additional attention during deployment. Grad-CAM explainability was used so that users could observe model-detected sick areas on plant leaves. And the prediction logic using uncertainty awareness has improved the reliability of the system and reduced overconfident predictions.

The following ethical considerations were addressed in the project:

- Responsible use of AI in agriculture.
- Improved transparency using Grad-CAM explainability.
- Reduction of misleading predictions through uncertainty-aware logic.
- Protection of uploaded image data and user privacy.
- AI system designed for assistance rather than complete dependency.

AI in agriculture should always be implemented based on ethical and responsible principles. On some occasions, AI outputs may be uncertain predictions due to image quality issues or the presence of disease symptoms that are not visible. Because of this, AGRO-GPT was not intended to be a substitute for farming experts but rather a tool that can be used by them.

Being able to visually inspect the contaminated areas after they have been marked by the users, the explainability feature is also a tool for increasing accountability. It fosters transparency in the decision-making process that is supported by AI and minimizes the issue of blind trust and reliance on automated forecasting.

5.2 Social Impact and Societal Relevance

The AGRO-GPT system would be helpful to the agricultural industry by providing a tool that can help farmers better manage crops and identify early signs of disease in their plants. Being able to identify signs of plant disease early on can prevent losses, increase production, and reduce damage to crops. And using the web browser, the user accesses the system.

This cloud-based deployment of the system allows one to do so without costly hardware or technical knowledge. It increases access to those researchers, students, and agricultural workers from urban and rural places.

The following social impacts are associated with the project:

- Improved awareness about plant diseases.
- Faster and easier disease diagnosis support.
- Increased accessibility to AI-powered agricultural tools.
- Support for farmers in reducing crop losses.
- Promotion of smart farming technologies.

5.3 Sustainability Consideration

The AGRO-GPT system is also able to enhance sustainability in agriculture by promoting early detection of disease and suitable treatment for the crops being cultivated. By enabling early diagnosis of disease, the system can contribute to reducing excessive use of pesticides and prevent a large percentage of crop failure.

Using cloud-based deployment and lightweight deep learning architecture for a scalable and energy-efficient solution also makes it feasible. The experiment demonstrates how artificial intelligence can be employed in agriculture to enhance sustainable practices and improve productivity.

The following sustainability considerations were addressed in the project:

- Support for sustainable and smart farming practices.

- Reduction of unnecessary pesticide usage.
- Early disease detection for minimizing crop loss.
- Efficient cloud-based deployment architecture.
- Scalable AI system for future agricultural applications.

Chapter 6

CONCLUSION AND FUTURE SCOPES

6.1 Summary of Findings

AGRO-GPT system beautifully demonstrated the deployment of Artificial Intelligence, Deep Learning, Explainable AI, and cloud computing technologies in the smart agriculture and plant disease detection sector. The platform brought together EfficientNet-B0 transfer learning model, Grad-CAM interpretability, uncertainty-aware prediction strategy, intelligent agricultural recommendation system, FastAPI backend integration, and cloud deployment under one AI-assisted agricultural support forum.

Training and testing results showed that the designed platform could reliably identify various plant diseases in numerous crop types with remarkable accuracy and very balanced performance metrics. The last EfficientNet-B0 model after training reached approximately 93.4% test accuracy on a balanced dataset of 14,836 images representing 42 disease classes. Other performance measures like precision recall F1-score, and AUC-ROC score also illustrated quite a consistent and dependable classification performance.

Possibly one of the most remarkable characteristics of the project was the introduction of the uncertainty-aware prediction mechanism. This capability improved the overall system reliability by determining low-confidence predictions and decreasing the probability of overconfident misclassification. With the use of confidence-aware prediction analysis, the AGRO-GPT platform has gained significant credibility and is highly suitable for real agricultural assistance application.

Not only did the project commit itself to agriculture, but also it dealt with human concern by launching Grad-CAM explainability visualization. Unlike typical AI black-box systems, AGRO-GPT incorporates explanation features with visual indication by showing the areas of plant leaves that resulted in the prediction. This attribute is definitely helping the AI disease

diagnosis systems to become transparent, interpretable, and creating user trust.

Besides its diagnostic function, the intelligent agricultural knowledge engine contributed greatly to the system getting closer to the real-world scenario by providing disease descriptions causes remedies, and prevention techniques and agricultural advice. Such characteristics enabled the system conversion from a mere image classification model to a complete agricultural platform.

6.2 Limitations of the Work

Despite really being the AGRO-GPT system proposed above, successfully detecting plant diseases and supporting agriculture, some limitations are still there. The quality and resolution of the images of the uploaded plant leaves influence the efficiency of the model greatly. Pictures taken with poor light or blurry images, with noisy backgrounds or partially captured leaves, are unable to predict correctly.

This system now makes training with 42 classes of disease and few types of crop. So, unknown diseases in training conditions cannot be detected. Also, some diseases that seem to be similar may make confused predictions in fields.

But some constraints, like dependence on internet access and short inference lag while the server is busy or the backend is setting up, are also pointed out by its implementation in the cloud. AI-based recommendations generated by a system, rather than substituting agricultural professional advice, in fact guide one as a support.

The following limitations were identified during the project:

- Prediction accuracy depends on image quality and lighting conditions.
- Limited number of crop categories and disease classes.
- Possible confusion between visually similar diseases.
- Dependence on internet connectivity for cloud deployment.
- AI recommendations should not replace professional agricultural advice.

Besides, the experiment reveals that explainable AI can increase users' trust in tools designed for agriculture. Unlike regular prediction systems without any explanation (black-box), AGRO-GPT improves the understanding of how the system works by giving graphical explanations and also predicting with a level of confidence. By using cloud deployment technologies, the access and scaling of the system for useful work in the real world are even made easier.

That method can in the end be merged with farming environments that are smart and equipped with the Internet of Things (IoT), drone-based crop monitoring, and satellite imaging. These enhancements can make AGRO-GPT a fully-fledged smart farming ecosystem capable of helping with large-scale crop monitoring and precision farming.

6.3 Scope for Future Enhancements

The suggested AGRO-GPT system can possibly be further optimized by including more crop types and combinations of diseases into the dataset for improved generality and practicality of the model. Future systems can also include more real-field data collected in a diverse range of environmental conditions.

To improve the accessibility of the current web-based platform, this system could be further developed into a mobile application. For remote applications, usability might be increased by additional functionalities like voice communication, local language, and offline prediction.

The following future enhancements can be considered for improving the AGRO-GPT system and expanding its practical usability in intelligent agriculture applications:

- **Addition of More Crop Categories and Disease Classes**

Right now, the system is capable of handling multiple crop categories and 42 disease classes. Including other crops, disease forms, and healthy plant states from different farming areas in future versions of the dataset are ways to increase the dataset size. A dataset that is bigger and more diverse will help the model to generalize better and will result in more accurate predictions when the model is used in real-world farming scenarios.

- **Development of Mobile Application Version**

Currently, AGRO-GPT is a web-based platform accessible via different browsers. Also, the next step for the project could be developing the Android and iOS mobile apps which would allow farmers to simply take leaf photos with their phones and receive instant disease analysis. It being on mobiles can improve the farmers' accessibility to the tool and their ability to have it with them while they are in the field.

- **Multilingual and Voice-Based Farmer Support**

Farmers are most likely not going to choose systems based on English at all. So, the upcoming versions of AGRO-GPT will support multiple regional languages. Beyond voice interaction which will be a bonus feature. This makes the platform more friendly to rural users and farmers might even enjoy the platform better and use it more efficiently.

- **Integration with IoT Sensors and Smart Farming Devices**

One of the future good improvements for the system is using Internet of Things (IoT)-enabled sensors. These sensors write planter's stories by detecting the changes in the soils like changes in the moisture level temperature humidity, and weather patterns. Besides making it easier to predict plant diseases, information from these environmental sensors can also improve agricultural advice and support precision farming practices.

- **Drone-Based Real-Time Crop Monitoring**

AGRO-GPT could be tied to drone technology and then work as a big farming surveillance tool. Drones equipped with high-resolution cameras can take aerial pictures of farms, which the system would afterward use for monitoring crop health and detecting the spread of diseases in real-time over wide farming areas.

- **Offline Prediction Support for Remote Areas**

Since the present cloud deployment is dependent on the internet, a user needs proper internet connectivity to avail the prediction services. A possible future enhancement is the development of efficient compact AI models that can operate on edge devices or mobile systems even in the absence of an internet connection. In this way the usability can be Really improved in remote rural regions where network connectivity is limited.

- **Advanced AI-Powered Agricultural Recommendation Systems**

The crop advisory system will be Much improved by using Large Language Models (LLMs) and smart retrieval-based mechanisms. Then, these systems can provide very personalized and extremely localized farm management directions based on the type of crop planted, level of pest/disease attack, local weather conditions, and farming records.

- **Integration of Weather Forecasting and Climate Analysis**

Later, we could bring in weather forecasting APIs and climate analysis to predict outbreaks of diseases based on environmental conditions. This would be of great help for farmers to take preventive measures before the diseases spread over their crops.

REFERENCES

- [1] Tan, M., & Le, Q. (2019), “*EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*”, Proceedings of ICML, pp. 6105–6114.
- [2] Selvaraju, R.R., et al. (2017), “*Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*”, Proceedings of ICCV, pp. 618–626.
- [3] Mohanty, S.P., Hughes, D.P., & Salathé, M. (2016), “*Using Deep Learning for Image-Based Plant Disease Detection*”, Frontiers in Plant Science, Vol. 7, pp. 1419.
- [4] Atila, Ü., et al. (2021), “*Plant Leaf Disease Classification using EfficientNet Deep Learning Model*”, Ecological Informatics, Vol. 61, pp. 101182.
- [5] FAO (2023), “*Crop Losses and Food Security Report*”, Food and Agriculture Organization.
- [6] Reddy, P.K., et al. (2024), “*Explainable AI for Plant Disease Detection using Grad-CAM*”, Proceedings of IEEE ICECA.
- [7] Jayaprakash Pondy (2023), “*Wheat Leaf Disease Dataset*”, Kaggle Dataset. Available at: <https://www.kaggle.com/datasets/jayaprakashpondy/wheat-leaf-disease>
- [8] Anshul M (2024), “*Rice Disease Dataset*”, Kaggle Dataset.
- [9] PlantVillage Dataset, Kaggle Dataset.
- [10] PlantDoc Dataset, Kaggle Dataset.
- [11] Too, E.C., Yujian, L., Njuki, S., & Yingchun, L. (2019), “*A Comparative Study of Fine-Tuning Deep Learning Models for Plant Disease Identification*”, Computers and Electronics in Agriculture, Vol. 161, pp. 272–279.
- [12] Ferentinos, K.P. (2018), “*Deep Learning Models for Plant Disease Detection and Diagnosis*”, Computers and Electronics in Agriculture, Vol. 145, pp. 311–318.
- [13] Sladojevic, S., Arsenovic, M., Anderla, A., Culibrk, D., & Stefanovic, D. (2016), “*Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification*”, Computational Intelligence and Neuroscience, Vol. 2016, pp. 1–11.

- [14] Brahim, M., Boukhalifa, K., & Moussaoui, A. (2017), “*Deep Learning for Tomato Diseases: Classification and Symptoms Visualization*”, Applied Artificial Intelligence, Vol. 31, No. 4, pp. 299–315.
- [15] Ramcharan, A., Baranowski, K., McCloskey, P., Ahmed, B., Legg, J., & Hughes, D.P. (2017), “*Deep Learning for Image-Based Cassava Disease Detection*”, Frontiers in Plant Science, Vol. 8, pp. 1852.
- [16] Kamilaris, A., & Prenafeta-Boldú, F.X. (2018), “*Deep Learning in Agriculture: A Survey*”, Computers and Electronics in Agriculture, Vol. 147, pp. 70–90.
- [17] Barbedo, J.G.A. (2018), “*Factors Influencing the Use of Deep Learning for Plant Disease Recognition*”, Biosystems Engineering, Vol. 172, pp. 84–91.
- [18] Dosovitskiy, A., et al. (2021), “*An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*”, Proceedings of ICLR.
- [19] Howard, A.G., et al. (2017), “*MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*”, arXiv preprint arXiv:1704.04861.
- [20] He, K., Zhang, X., Ren, S., & Sun, J. (2016), “*Deep Residual Learning for Image Recognition*”, Proceedings of CVPR, pp. 770–778.
- [21] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K.Q. (2017), “*Densely Connected Convolutional Networks*”, Proceedings of CVPR, pp. 4700–4708.
- [22] Chollet, F. (2017), “*Xception: Deep Learning with Depthwise Separable Convolutions*”, Proceedings of CVPR, pp. 1251–1258.
- [23] Goodfellow, I., Bengio, Y., & Courville, A. (2016), “*Deep Learning*”, MIT Press, Cambridge, Massachusetts.
- [24] Lundberg, S.M., & Lee, S.I. (2017), “*A Unified Approach to Interpreting Model Predictions*”, Proceedings of NeurIPS, pp. 4765–4774.
- [25] Simonyan, K., & Zisserman, A. (2015), “*Very Deep Convolutional Networks for Large-Scale Image Recognition*”, Proceedings of ICLR.

APPENDIX

Project Proposal

Title: AGRO-GPT: AI-Based Plant Disease Detection and Smart Agricultural Assistance System

Objective: To develop an intelligent agricultural assistance platform that combines plant disease detection, explainable AI visualization, uncertainty-aware prediction logic, and AI-powered agricultural recommendations into a unified system.

Scope:

- To develop a deep learning based plant disease classification system using EfficientNet-B0.
- To train the model on real-world multi-crop plant disease datasets.
- To integrate Grad-CAM explainability for visualization of infected leaf regions.
- To implement uncertainty-aware prediction logic for reliable disease classification.
- To develop AI-powered agricultural recommendation and disease guidance modules.
- To integrate FastAPI backend services and React frontend interface.
- To deploy the complete system using Hugging Face Spaces and Vercel.

Expected Outcomes:

- Accurate plant disease classification across 42 disease classes.
- Approximately 93% test accuracy using EfficientNet-B0 architecture.
- Explainable AI visualization using Grad-CAM heatmaps.
- Real-time disease prediction through cloud deployment.
- AI-generated agricultural recommendations and disease guidance.
- User-friendly web-based agricultural assistance platform.

Similarity Check Report

Team Contribution

Name	Contribution
Ansuman Mantri	Implementation of EfficientNet-B0 based plant disease detection model, Grad-CAM explainability integration, uncertainty-aware prediction logic, FastAPI backend development, Hugging Face deployment, and overall system integration.
Aniket Nayak	Frontend development using React + Vite, API integration between frontend and backend, user interface design, testing, debugging, and Vercel deployment configuration.
Dipankar Mohapatra	Dataset collection, preprocessing, exploratory data analysis (EDA), image augmentation, dataset balancing, visualization generation, and performance analysis.
Mrutyunjaya Tripathy	Literature survey, documentation preparation, result interpretation, report formatting, deployment testing, and project presentation support.

Course Outcomes (COs), Program Outcomes (POs) and Program Specific Outcomes (PSOs)

Course Outcomes (COs)

- CO1:** Apply engineering knowledge and computing fundamentals to analyze complex real-world problems, identify requirements, and define appropriate project objectives.
- CO2:** Design and develop end-to-end software or computing solutions using appropriate algorithms, architectures, and modern tools to meet specified constraints.
- CO3:** Conduct investigations, experimentation, and performance evaluation using suitable metrics, datasets, simulations, or prototypes to validate project outcomes.
- CO4:** Demonstrate the ability to function effectively as an individual and as a team member/leader, managing tasks, timelines, and resources in multidisciplinary environments.
- CO5:** Communicate technical ideas, design decisions, and results effectively through well-structured project reports, presentations, and demonstrations.
- CO6:** Recognize and apply professional ethics, societal impact, sustainability considerations, and lifelong learning principles in project development and execution.

Program Outcomes (POs)

- PO1: Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
- PO2: Problem analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and engineering sciences.
- PO3: Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet specified needs with appropriate consideration for public health and safety, and cultural, societal and environmental considerations.
- PO4: Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of information to provide valid conclusions.

PO5: Modern tool usage: Create, select and apply appropriate techniques, resources and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6: The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to professional engineering practice.

PO7: Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts and demonstrate knowledge of, and need for, sustainable development.

PO8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO9: Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams and in multidisciplinary settings.

PO10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large.

PO11: Project management and finance: Demonstrate knowledge and understanding of engineering and management principles and apply these to one's own work as a member and leader in a team to manage projects in multidisciplinary environments.

Program Specific Outcomes (PSOs)

PSO1: The ability to understand and develop computer programs in the areas related to business intelligence, big data analytics, web design and networking for efficient design of computer-based system of varying complexity.

PSO2: The ability to apply standard practices and strategy in software project development using open ended programming environments to deliver a quality product for business success.

Evidences Linked to CO–PO & PSO Mapping

COs	Mapped POs and PSOs	Evidence in Project Work	
		Description	Section with Evidence
CO1	PO1 (Engineering Knowledge)	1. Development of EfficientNet-B0 based plant disease classification model. 2. Implementation of transfer learning for agricultural image classification. 3. Integration of uncertainty-aware prediction logic.	Section 3.3, 3.4, 4.2
	PO2 (Problem Analysis)	1. Identification of plant disease classification challenges. 2. Dataset preprocessing and balancing techniques. 3. Comparative evaluation using classification metrics.	Section 3.1, 3.5, 4.3
	PSO1	1. Development of AI-based agricultural assistance system. 2. Integration of intelligent disease recommendation engine. 3. Deployment of cloud-based prediction platform.	Section 4.1, 4.4, 4.6
CO2	PO3 (Design/Development of Solutions)	1. Design of modular AI-based disease detection architecture. 2. Development of frontend-backend integrated workflow. 3. Implementation of real-time prediction system.	Section 3.2, Figure 3.1, 4.5
	PO5 (Engineering Tool Usage)	1. Implementation using PyTorch and TIMM frameworks. 2. Use of FastAPI and React for deployment integration. 3. Utilization of Grad-CAM explainability tools.	Section 3.6, 4.2, 4.5
	PSO1	1. Development of explainable agricultural AI platform. 2. Real-time cloud deployment using Hugging Face and Vercel. 3. Practical implementation of intelligent agriculture tools.	Section 4.5, 4.6
CO3	PO2 (Problem Analysis)	1. Analysis of dataset imbalance and noisy field images. 2. Improvement of model generalization capability. 3. Evaluation using confusion matrix and performance metrics.	Section 3.1, 4.3
	PO4 (Investigation of Complex Problems)	1. Experimental analysis of EfficientNet-B0 performance. 2. Study of Grad-CAM explainability outputs. 3. Comparative analysis of prediction confidence.	Section 4.2, Figure 4.3

Continued on next page

COs	Mapped POs and PSOs	Description	Section with Evidence
	PO4 (Investigation of Complex Problems)	1. Hyperparameter tuning and model optimization. 2. Visualization-based analysis using EDA plots. 3. Validation using multiple evaluation metrics.	Section 3.5, 4.3
	PSO1	1. Integration of AI recommendation system for agriculture. 2. Intelligent disease guidance and prevention suggestions. 3. Practical deployment for real-world agricultural support.	Section 4.4, 4.6
	PSO2	1. Development of scalable AI deployment workflow. 2. Cloud-based frontend and backend integration. 3. Real-time prediction and visualization generation.	Section 4.5, app.py
CO4	PO8 (Individual & Collaborative Team Work)	1. Collaborative development of frontend and backend modules. 2. Team-based dataset preparation and documentation work. 3. Coordinated deployment and testing activities.	Appendix, Team Contribution
	PO10 (Project Management & Finance)	1. Planning and execution of cloud deployment workflow. 2. Efficient utilization of open-source AI frameworks. 3. Project timeline and module management.	Section 4.5, Appendix
	PSO2	1. Integration of multiple AI modules into unified system. 2. Deployment-oriented software engineering practices. 3. Management of real-time AI inference workflow.	Section 4.5, 4.6
CO5	PO9 (Communication)	1. Visualization of Grad-CAM heatmaps for explainability. 2. User-friendly frontend interface development. 3. Presentation of disease recommendations and outputs.	Section 4.4, Figure 4.2
CO6	PO6 (The Engineer and the World)	1. AI-based support system for improving agricultural productivity. 2. Early disease detection to reduce crop loss. 3. Promotion of intelligent farming practices.	Section 5.2, 5.3
	PO7 (Ethics)	1. Responsible usage of AI-based predictions. 2. Transparency through Grad-CAM explainability. 3. Ethical deployment of agricultural recommendation systems.	Section 5.1

Continued on next page

COs	Mapped POs and PSOs	Description	Section with Evidence
	PO11 (Life-Long Learning)	1. Exploration of Explainable AI and transfer learning concepts. 2. Learning cloud deployment and API integration techniques. 3. Practical implementation of modern AI frameworks.	Section 4.5, Appendix

Code Structure

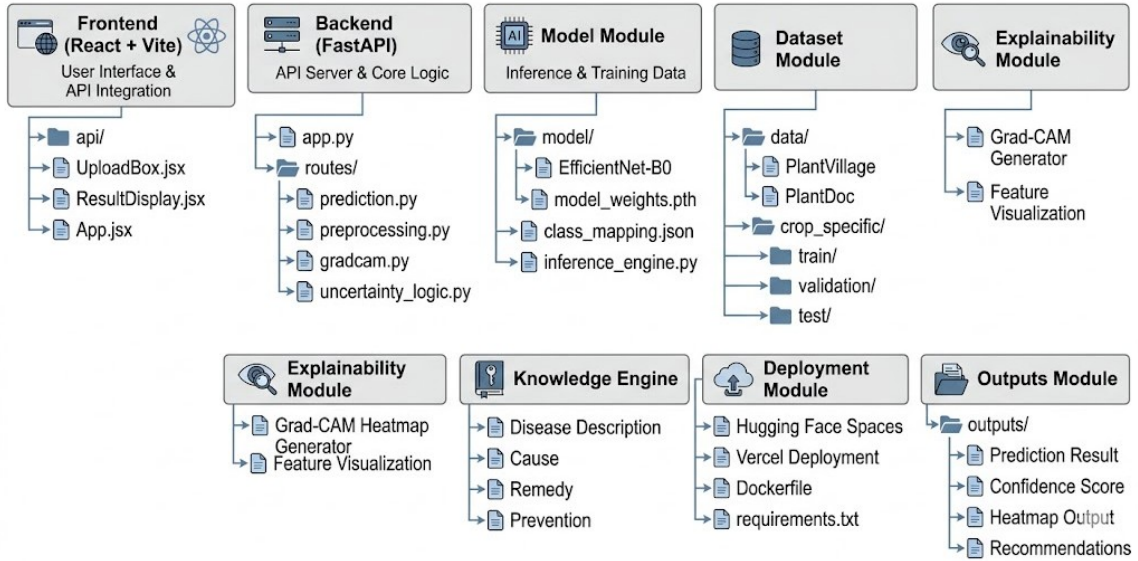


Figure 6.1: Code Structure Overview of AGRO-GPT System

The AGRO-GPT system has its code structure in a modular form and well arranged. It separates various roles into independent modules like frontend services, backend APIs, deep learning modules, explainability generation, preprocessing utilities, deployment configuration, and output management. By using this modular design the system in total is more scalable maintainable and easier to debug plus have more extensibility in the future.

React and Vite were used to develop the frontend module. It manages the user interface aspects, such as enabling users to interact with the system, upload images, display results, and make calls to backend APIs. Actually, the frontend offers users with a convenient and flexible way to upload their pictures of plant leaves. After that, they can view AI predictions, Grad-CAM images, and the system's agricultural recommendations.

FastAPI was used for the backend module development, so it acts as the primary communication channel for the entire AGRO-GPT platform. It includes support for different features such as API routes, image prerequisite transformation, as well as the capability of issuing prediction, Grad-CAM, and uncertainty-aware prediction analysis. The trained EfficientNet-B0 model is connected to this module. In addition, backend is the one who mediates the communication between frontend services and AI inference modules.

The model module consists of the trained model EfficientNet-B0 architecture, model weights, inference engine, disease classification mapping files, etc. This module is the key element for feature extraction, disease prediction, and confidence score generation. Splitting model-related components not

only allows better reuse of components but also makes it easier to upgrade or retrain models later on.

The dataset module manually organizes all agricultural datasets into different levels of the data hierarchy i.e. training, validation, and testing folders. A well-structured dataset leads to easier preprocessing consistency and overall easier dataset handling during model development and testing.

The explainability module performs Grad-CAM visualization methods that produce heatmaps indicating infected areas of the leaves. This module improves model interpretability and allows the user to visually understand which leaf areas in the image led to the prediction decision.

The knowledge engine module is the agricultural domain knowledge base that keeps disease-related information like descriptions causes remedies, prevention methods, and agricultural tips of various crop diseases. This module, by converting the AGRO-GPT system from a mere classification platform to an intelligent agricultural assistance system, adds great value to the project.

deployment module comprises cloud deployment configuration files including Hugging Face deployment setup, Vercel frontend deployment configuration, Docker-related settings, and dependency management files. These deployment tools simplify the process of system hosting and remote accessibility.

outputs module is mainly involved in storing prediction results, generated Grad-CAM heatmaps, confidence scores, and recommendation outputs. Separating outputs not only improves traceability but is also very useful for monitoring and debugging during real-time inference operations.

To sum up, the modular code structure of AGRO-GPT has a great impact on the improvement of software engineering quality, system reliability scalability collaborative development, and maintainability of the AI-based agricultural assistance platform in the long term.

Code Listings

6.3.1 Data Preprocessing and Augmentation

```
train_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225])])
val_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225])])
train_dataset = datasets.ImageFolder(
    train_dir,
    transform=train_transforms
)
val_dataset = datasets.ImageFolder(
    val_dir,
    transform=val_transforms
)
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=32,
    shuffle=False
)
```

Figure 6.2: Data Preprocessing and Augmentation Pipeline

6.3.2 EfficientNet-B0 Model Training

```
model = timm.create_model(
    "efficientnet_b0",
    pretrained=True,
    num_classes=42
)
model = model.to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {running_loss:.4f}")

torch.save(model.state_dict(), "efficientnet_b0.pth")
```

Figure 6.3: EfficientNet-B0 Training Implementation

6.3.3 Prediction and Uncertainty-Aware Logic

```
def predict_disease(image, model, class_names):  
    model.eval()  
  
    image = preprocess_image(image)  
    image = image.unsqueeze(0).to(device)  
  
    with torch.no_grad():  
        outputs = model(image)  
        probabilities = torch.softmax(outputs, dim=1)  
        confidence, predicted = torch.max(probabilities, 1)  
  
    confidence_score = confidence.item() * 100  
  
    predicted_class = class_names[predicted.item()]  
  
    if confidence_score < 60:  
        status = "Uncertain Prediction"  
    else:  
        status = "Confident Prediction"  
  
    return {  
        "disease": predicted_class,  
        "confidence": confidence_score,  
        "status": status  
    }
```

Figure 6.4: Prediction and Confidence Evaluation Logic

6.3.4 Grad-CAM Explainability Module

```
def generate_gradcam(model, image_tensor, target_layer):  
    cam = GradCAM(  
        model=model,  
        target_layers=[target_layer]  
    )  
  
    grayscale_cam = cam(  
        input_tensor=image_tensor  
    )[0]  
  
    rgb_image = image_tensor.squeeze().permute(1, 2, 0).cpu().numpy()  
  
    visualization = show_cam_on_image(  
        rgb_image,  
        grayscale_cam,  
        use_rgb=True  
    )  
  
    heatmap_path = "gradcam_output.jpg"  
  
    cv2.imwrite(heatmap_path, visualization)  
  
    return heatmap_path
```

Figure 6.5: Grad-CAM Heatmap Generation Function

6.3.5 FastAPI Backend Integration

```
app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    image = Image.open(file.file).convert("RGB")
    prediction = predict_disease(
        image,
        model,
        class_names
    )
    gradcam_image = generate_gradcam(
        model,
        preprocess_image(image).unsqueeze(0).to(device),
        target_layer=model.conv_head
    )
    disease_info = disease_database[
        prediction["disease"]
    ]
    return {
        "prediction": prediction,
        "gradcam": gradcam_image,
        "description": disease_info["description"],
        "cause": disease_info["cause"],
        "remedy": disease_info["remedy"],
        "prevention": disease_info["prevention"]
    }
```

Figure 6.6: FastAPI Backend Prediction API